

# Empowering Users with Algorithmic Control and Transparency in Tourism Recommender Systems: A DSA-Compliant Approach

DAVID GALLOB, Technical University of Munich, Germany

The recommender systems people encounter in daily life are very often "black boxes", which means that users rarely know how their recommendations are generated and they have no control over which items are recommended. In tourism, this leads to a lack of trust and user satisfaction, as users can never be sure if their recommendation is really optimal or influenced by profit maximization goals of the platform. The EU has therefore introduced the Digital Services Act (DSA), a framework which requires platforms to explain their algorithms in an understandable way and offer users a profiling-free alternative. In this paper a web-based prototype for hotel recommendations which addresses these issues by implementing two algorithms is presented. The web-app offers detailed explanations of the underlying decision algorithms and shows for each result, how the recommendation was computed. The two algorithms which come to use are Weighted Multi-Attribute Utility Theory (MAUT), which lets users set weights on the different attributes (e.g. stars, distance to center, price) and Constraint Filtering, where the user sets hard limits to the attributes (e.g. price must be below 100 Euro per night). This paper details the system architecture as well as the mathematical foundations of the used algorithms along with the UI features explaining the algorithms to the user.

Additional Key Words and Phrases: Recommender Systems, User Control, Multi-Attribute Utility Theory, Constraint Filtering, Digital Services Act, Explainable AI, E-Tourism

## 1 Introduction

In the day-to-day life, recommender systems (RS) are ubiquitous from online shopping and video streaming to hotel and flight recommendation platforms [1]. Many prominent RS do not let users know how their recommendations are generated and offer no direct means of influencing the outcome [2]. Those systems are often built on complex algorithms (mainly neural networks) which process enormous amounts of user to generate suggestions. They rarely explain users their underlying logic. This lack of transparency can hinder trust and user satisfaction and leads to frustration and cognitive overhead. The tourism domain where a high financial expenditure and the emotional investment is generally high is particularly affected by this issue [8].

The Human-Computer Interaction (HCI) field has focused their research on user influence and transparency (explainability) to address these shortcomings [6, 12]. Instead of using automated profiling, interactive recommender systems give users the ability to influence the recommendation process by adjusting sliders which represent their importance or setting hard constraints for the attributes (e.g., price or distance).

The demand for transparency is further fueled by the European Union's Digital Services Act (DSA), which governs online platforms and aims to protect user's rights. The DSA mandates that online platforms must provide users with transparent information about the main parameters of their recommender systems in an understandable language in plain text and offer options to influence them (Article 27) [3]. Article 38 further obliges very large online platforms (VLOPs) to provide users with at least one option for their recommender systems that is not based on profiling. These articles require developers to shift from black-box profiling towards transparent and user controlled platforms.

In this paper the design decisions, architecture and implementation of a web-application tourism recommender system prototype which is compliant with the DSA is detailed. It addresses the transparency as well as the user influence goals by implementing two algorithms:

---

Author's Contact Information: David Gallob, david.gallob@tum.de, Technical University of Munich, School of Computation, Information and Technology, Munich, Germany.

---

2026. ACM 2476-1249/2026/6-ART1

<https://doi.org/>

- (1) **Algorithm A: Weighted Multi-Attribute Utility Theory (MAUT):** This algorithm uses sliders to determine weights for hotel attributes (price, location, star rating, customer rating, and public transit access). The algorithm performs a normalization of the attributes and computes a weighted utility score for each hotel. Finally the hotels are ranked according to their utility score (percentage match).
- (2) **Algorithm B: Constraint-Based Filtering:** In Constraint-Based Filtering the user defines hard limits for each attribute (e.g. maximum price or minimum rating). All hotels which do not meet the constraints are filtered out. This allows users to specify their preferences in a clear and understandable way without the need for complex mathematical calculations.

The prototype features a banner at the top of the landing page explaining the chosen algorithm in clear words and offers a button which on-click shows the mathematical formula of the chosen algorithm for enhanced transparency. The calculated results also offer a collapsible mathematical breakdown (only when activated to not overwhelm not interested users) showing a step-by-step calculation how the result's score was computed. This allows users to understand how the results are generated and how their inputs influence the outcome. The calculation breakdown is generated using KaTeX and renders the exact mathematical formulas and normalization equations used in the calculations, allowing users to trace how their inputs directly translate into the presented results.

The remainder holds the following structure: In [section 2](#) the related work in current tourism recommender systems, MAUT and explanation interfaces is identified. [section 3](#) details the system's architecture and the mock dataset. [section 4](#) shows the mathematical foundations of the two used algorithms. [section 5](#) details the UI design and transparency features. [section 7](#) proposes a protocol for a future user study and extensions, and [section 8](#) concludes the paper. In the Appendix the complete mock database used in the project, followed by the author contribution table and GenAI disclosures are attached.

## 2 Related Work

The development of recommender systems in tourism is characterized by unique challenges that distinguish it from other application domains, such as movie or music recommendation [2]. Tourism decisions are high-involvement, occasional, and multi-attribute in nature [8]. Unlike low-cost consumer goods, booking a hotel or planning a vacation involves substantial financial risk, time commitment, and emotional expectation. Consequently, users are rarely satisfied with simple "top-N" recommendations; they require extensive information, comparison tools, and the ability to express specific, situational constraints.

Traditional tourism recommender systems (TRSs) are often categorized into collaborative filtering, content-based filtering, and hybrid approaches [4, 9]. While collaborative filtering leverages the historical ratings of similar users, it suffers from severe cold-start problems, as new hotels lack reviews, and occasional travelers lack historical profiles. Content-based systems match hotel features with user preference profiles, but they frequently struggle with explainability and cannot easily adapt to sudden shifts in user needs (e.g., a business traveler requiring a central location vs. a family seeking budget accommodations).

To overcome these limitations, multi-attribute decision-making frameworks, particularly Multi-Attribute Utility Theory (MAUT), have been applied to interactive recommendation [10]. MAUT provides a structured mathematical methodology for evaluating alternatives based on multiple, potentially conflicting criteria [5]. In a MAUT-based recommender, the system defines a utility function for each attribute (e.g., mapping price to a utility between 0 and 1) and aggregates these utilities using a weighted sum. Schmitt et al. [10] demonstrated that MAUT is highly suited for conversational or interactive recommenders, as it maps directly to human decision-making processes. MAUT is fundamentally compensatory: a deficiency in one attribute (e.g., a high price) can be offset by an exceptionally high value in another (e.g., a prime location).

In contrast, constraint-based recommendation represents a non-compensatory approach. Instead of calculating a continuous score, constraint-based systems utilize explicit knowledge rules to filter out items that violate user-defined thresholds. This approach is highly predictable, transparent, and does not require historical user profiling, aligning perfectly with Article 38 of the DSA [3].

Prior research emphasizes that the success of interactive recommender systems depends heavily on their user interfaces [7, 11]. Providing explanation interfaces not only improves user understanding but also increases their trust and perceived control over the system [12]. Knijnenburg et al. [6] proposed a comprehensive framework showing that user control, when paired with high explainability, leads to higher satisfaction and system adoption. The integration of real-time sliders and mathematical breakdowns investigated in this work directly builds on these findings, operationalizing regulatory requirements into specific user interface patterns.

### 3 System Architecture and Implementation

The prototype is designed as a client-side, responsive single-page application (SPA). The technical stack consists of React 18, Vite for the build system, and vanilla CSS for layout and styling. Client-side math rendering is powered by KaTeX, a fast LaTeX math rendering library, integrated via a custom React wrapper.

The system architecture is structured to support real-time user interaction. Figure 1 illustrates the flow of data through the system.

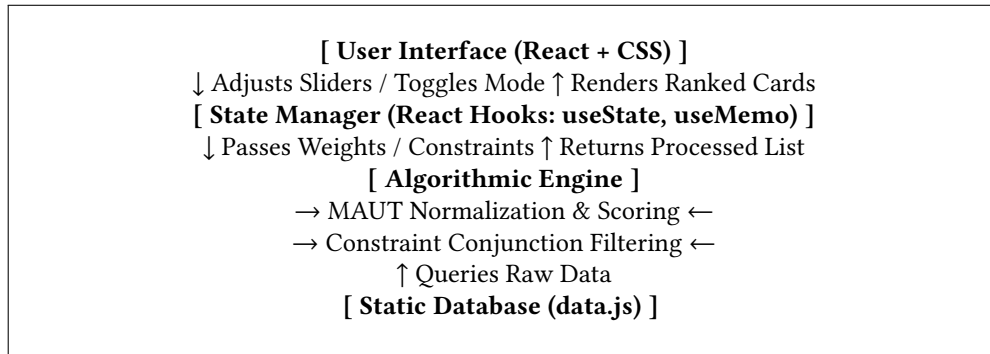


Fig. 1. Data Flow and Component Architecture of the Recommender Prototype.

The application state manages:

- The selected city ( $c \in \{\text{Vienna, Munich}\}$ ).
- The active algorithmic mode ( $m \in \{\text{Weighted MAUT, Constraint-Guided Filtering}\}$ ).
- The five attribute weights ( $w_i \in [0, 100]$ ) for the MAUT mode.
- The five attribute bounds ( $b_i$ ) for the Constraint Filtering mode.
- The expanded state of individual hotel cards (indicating whether the mathematical breakdown panel is visible).
- The global formula toggle (controlling whether plain-language explanations or mathematical LaTeX formulas are shown in the transparency banners).

The dataset, stored in a static module ('data.js'), comprises 20 hotels (10 in Vienna and 10 in Munich). The hotels were designed to represent a realistic and diverse spectrum of accommodations, ranging from budget-friendly youth hostels to luxury five-star resorts. Each hotel  $X_j$  is defined by a unique identifier, name, city, description, and five key quantitative attributes:

- (1) **Price per Night ( $p_j$ )**: Measured in USD (\$). Ranging from \$35 (Hostel Vienna West) to \$550 (Mandarin Oriental Munich).
- (2) **Distance to Center ( $d_j$ )**: Measured in kilometers (km) from the historical city center (Stephansplatz in Vienna, Marienplatz in Munich). Ranging from 0.1 km (Stephansplatz) to 28.0 km (Airport Express Inn Munich).
- (3) **Star Rating ( $s_j$ )**: Ranging from 1 (basic tourist class) to 5 stars (luxury class).
- (4) **Guest Review Score ( $r_j$ )**: A continuous rating between 1.0 (worst) and 10.0 (excellent), representing aggregated guest satisfaction.
- (5) **Public Transport Access Score ( $t_j$ )**: A rating between 1.0 (poor) and 10.0 (excellent), representing proximity to subway stations, tram lines, and frequency of transit services.

By using ‘useMemo’, the application recalculates recommendations in real time whenever the user modifies a slider or toggles a control. Because the calculations are performed client-side on a bounded dataset, updates occur instantly (< 16 ms), providing immediate visual feedback.

## 4 Algorithmic Framework

To support the user-controlled interaction model, we formalize the mathematical models behind the two algorithmic recommendation modes.

### 4.1 Algorithm A: Weighted Multi-Attribute Utility Theory (MAUT)

Weighted MAUT evaluates each hotel  $X_j$  in the active city set  $H_{\text{city}}$  by mapping its raw attribute values to normalized utility scores and computing a weighted average. The calculation proceeds in three stages:

**4.1.1 Normalization of Attributes.** Since attributes are measured in different units (USD, kilometers, star counts, and continuous scales), they must be normalized to a standard utility scale  $u_i(x) \in [0, 1]$ , where 0 represents the least desirable outcome and 1 represents the most desirable.

The attributes are categorized into two types: cost metrics (where lower values are better) and benefit metrics (where higher values are better). Furthermore, the system employs two different normalization boundaries:

- **Relative Normalization:** Normalizes values based on the maximum and minimum values present within the active city’s dataset, maximizing the contrast between local options.
- **Absolute Normalization:** Normalizes values based on a fixed, theoretically defined range (e.g., 1 to 5 stars, or 1 to 10 rating scale).

The normalization functions for the five attributes are defined as follows:

- (1) **Price ( $u_{\text{price}}$ )**: A cost metric normalized relatively.

$$u_{\text{price}}(p_j) = \frac{p_{\max} - p_j}{p_{\max} - p_{\min}} \quad (1)$$

where  $p_{\max} = \max_{k \in H_{\text{city}}} p_k$  and  $p_{\min} = \min_{k \in H_{\text{city}}} p_k$ .

- (2) **Distance to Center ( $u_{\text{location}}$ )**: A cost metric normalized relatively.

$$u_{\text{location}}(d_j) = \frac{d_{\max} - d_j}{d_{\max} - d_{\min}} \quad (2)$$

where  $d_{\max} = \max_{k \in H_{\text{city}}} d_k$  and  $d_{\min} = \min_{k \in H_{\text{city}}} d_k$ .

- (3) **Star Rating ( $u_{\text{stars}}$ )**: A benefit metric normalized absolutely.

$$u_{\text{stars}}(s_j) = \frac{s_j - s_{\text{abs,min}}}{s_{\text{abs,max}} - s_{\text{abs,min}}} = \frac{s_j - 1}{4} \quad (3)$$

where  $s_j \in [1, 5]$ .

(4) **Guest Review Score** ( $u_{\text{rating}}$ ): A benefit metric normalized absolutely.

$$u_{\text{rating}}(r_j) = \frac{r_j - r_{\text{abs,min}}}{r_{\text{abs,max}} - r_{\text{abs,min}}} = \frac{r_j - 1}{9} \quad (4)$$

where  $r_j \in [1, 10]$ .

(5) **Public Transport Score** ( $u_{\text{transport}}$ ): A benefit metric normalized absolutely.

$$u_{\text{transport}}(t_j) = \frac{t_j - t_{\text{abs,min}}}{t_{\text{abs,max}} - t_{\text{abs,min}}} = \frac{t_j - 1}{9} \quad (5)$$

where  $t_j \in [1, 10]$ .

**4.1.2 Weighted Aggregation.** Let  $W = (w_{\text{price}}, w_{\text{location}}, w_{\text{stars}}, w_{\text{rating}}, w_{\text{transport}})$  be the vector of weights specified by the user, where each weight  $w_i \in [0, 100]$ . The total utility  $U(X_j)$  for hotel  $X_j$  is calculated as:

$$U(X_j) = \frac{\sum_{i \in \text{Attrs}} w_i \cdot u_i(x_{ij})}{\sum_{i \in \text{Attrs}} w_i} \times 100\% \quad (6)$$

where  $\text{Attrs} = \{\text{price}, \text{location}, \text{stars}, \text{rating}, \text{transport}\}$  and  $x_{ij}$  represents the raw value of attribute  $i$  for hotel  $j$ . The resulting utility  $U(X_j) \in [0, 100]\%$  represents the percentage match score. Hotels are sorted and presented to the user in descending order of  $U(X_j)$ .

## 4.2 Algorithm B: Constraint-Based Filtering

Constraint-Based Filtering does not calculate a numerical match score or rank hotels by preference. Instead, it screens the active city's dataset against five user-defined limits. Let  $C$  be the constraint tuple:

$$C = (P_{\text{max}}, D_{\text{max}}, S_{\text{min}}, R_{\text{min}}, T_{\text{min}}) \quad (7)$$

where:

- $P_{\text{max}}$  is the maximum price per night ( $P_{\text{max}} \in [\$30, \$600]$ ).
- $D_{\text{max}}$  is the maximum distance from the center ( $D_{\text{max}} \in [0.1, 30]$  km).
- $S_{\text{min}}$  is the minimum star rating ( $S_{\text{min}} \in [1, 5]$ ).
- $R_{\text{min}}$  is the minimum guest rating ( $R_{\text{min}} \in [1.0, 10.0]$ ).
- $T_{\text{min}}$  is the minimum public transit score ( $T_{\text{min}} \in [1.0, 10.0]$ ).

A hotel  $X_j$  is included in the recommended set  $R_{\text{filtered}}$  if and only if it satisfies the conjunction of all active constraints:

$$R_{\text{filtered}} = \{X_j \in H_{\text{city}} \mid f_{\text{filter}}(X_j) = \text{True}\} \quad (8)$$

where the filter function  $f_{\text{filter}}(X_j)$  is defined as:

$$f_{\text{filter}}(X_j) = (p_j \leq P_{\text{max}}) \wedge (d_j \leq D_{\text{max}}) \wedge (s_j \geq S_{\text{min}}) \wedge (r_j \geq R_{\text{min}}) \wedge (t_j \geq T_{\text{min}}) \quad (9)$$

This formulation represents a non-compensatory filtering logic. A hotel that violates even a single constraint (such as exceeding the maximum price by \$5) is immediately excluded from the results, regardless of whether it possesses perfect scores (e.g., 5 stars, 10.0 user rating) in all other categories.

## 4.3 Comparison of Decision Logic

The two models represent fundamentally different decision-making styles, as summarized in Table 1.

Table 1. Comparison of the Implemented Algorithmic Paradigms

| Dimension                   | Algorithm A: Weighted MAUT                         | Algorithm B: Constraint Filtering               |
|-----------------------------|----------------------------------------------------|-------------------------------------------------|
| <b>Decision Logic</b>       | Compensatory (trade-offs allowed)                  | Non-compensatory (hard boundaries)              |
| <b>Output Type</b>          | Ranked list with continuous match scores           | Unordered subset of matching items              |
| <b>User Controls</b>        | Relative importance weights ( $w_i \in [0, 100]$ ) | Threshold limits ( $P_{\max}, D_{\max}$ , etc.) |
| <b>Cognitive Style</b>      | Optimizing                                         | Screening / Satisficing                         |
| <b>DSA Compliance</b>       | Explains weights                                   | Explains inclusion/exclusion criteria           |
| <b>Profiling Dependency</b> | None (Profiling-free, real-time input)             | None (Profiling-free, real-time input)          |

## 5 User Interface Design and Explainability

The prototype interface was designed with a glassmorphism aesthetic (utilizing translucent backdrops, vibrant background gradients, and smooth animations) to create a premium, modern user experience. More importantly, the interface was structured to maximize explainability and user control.

### 5.1 DSA-Compliant Explanation Interfaces

To comply with the requirements of the Digital Services Act, transparency mechanisms are built directly into the primary interaction flow rather than being hidden in auxiliary pages.

**5.1.1 Global Explanation Banners.** At the top of the interface, a dedicated, colored information card dynamically adapts to the selected algorithm.

- In **Weighted MAUT** mode, the banner displays a blue theme, introducing the concept of multi-attribute weighting.
- In **Constraint-Guided** mode, the banner displays a green theme, introducing the logic of set reduction and non-compensatory filtering.

Both banners include a "Show Mathematical Details" toggle button. Clicking this button expands a sub-panel that renders the mathematical equations in LaTeX (Equation 6 for MAUT, and Equation 9 for Constraint-Guided Filtering) alongside a list of definitions. This design provides a tiered explanation model: novice users receive a brief, plain-language summary of how the parameters are aggregated, while expert users or regulators can inspect the exact mathematical formulas.

**5.1.2 On-Demand Collapsible Math Breakdowns.** For the Weighted MAUT mode, each hotel card features a "Show Mathematical Breakdown" button. Expanding this panel reveals a step-by-step trace of the utility calculation for that specific hotel. The panel displays:

- The general formula:  $\text{Score} = \frac{\sum (w_i \cdot \text{norm}_i)}{\sum w_i}$
- A tabular breakdown for each of the five attributes, displaying the normalized utility score ( $u_i$ ), the user-defined weight ( $w_i$ ), and the product ( $w_i \cdot u_i$ ).
- The summation step, showing the sum of products divided by the sum of weights.
- The final rounded match percentage.

For example, if a user selects Vienna and sets all weights to 50, and inspects the "Boutique Hotel am Stephansplatz" card, the breakdown displays:

- Price utility:  $0.86 \times 50 = 43$
- Location utility:  $1.00 \times 50 = 50$
- Stars utility:  $0.75 \times 50 = 38$

- Rating utility:  $0.91 \times 50 = 46$
- Transport utility:  $1.00 \times 50 = 50$
- Sum of products: 227
- Total weight: 250
- Final Match:  $\frac{227}{250} \times 100\% = 91\%$

This feature provides absolute transparency, eliminating the perception of the recommender system as a black box. The user can see exactly why a hotel is ranked first or second, and can directly trace the mathematical impact of adjusting a slider.

## 6 Implementation Challenges & Mathematical Edge Cases

During the development of the prototype, several critical mathematical bugs were identified, particularly within the MAUT calculation engine. These bugs are characteristic of interactive multi-attribute systems that rely on dynamic user inputs. This section details these challenges and their mathematical resolutions.

### 6.1 The Division-by-Zero Singularity on Zero Weights

**6.1.1 The Problem.** In early implementations of the Weighted MAUT algorithm, the denominator of the aggregation function was the sum of the user-defined weights,  $\sum_{i=1}^5 w_i$ . In an interactive interface, users are free to move all sliders to 0. If a user set all weights to 0, the sum of weights became 0, leading to a division-by-zero error:

$$U(X_j) = \frac{\sum_{i=1}^5 0 \cdot u_i(x_{ij})}{0} = \frac{0}{0} \quad (10)$$

In JavaScript, this evaluation resulted in 'NaN' (Not a Number), causing the hotel cards to fail to render, crashing the results grid, or rendering blank match badges.

**6.1.2 The Resolution.** To ensure mathematical robustness, a fallback mechanism was introduced. The total weight calculation in the codebase was modified using a logical OR operator: “`javascript const totalWeight = weights.price + weights.location + weights.stars + weights.rating + weights.publicTransport || 1;`” If the sum of weights evaluates to 0, the code defaults the denominator to 1. This prevents the application from crashing. However, if the total weight is 1 and all individual weights are 0, the raw score sum remains 0, yielding a match percentage of:

$$U(X_j) = \frac{0}{1} \times 100\% = 0\% \quad (11)$$

To improve the user experience, a preferred scientific alternative is to treat the case where all weights are zero as an “equal weighting” scenario, where each attribute is automatically assigned an equal weight of 20 (resulting in a default equal aggregation). This is implemented as: “`javascript const weightSum = weights.price + weights.location + weights.stars + weights.rating + weights.publicTransport; const totalWeight = weightSum === 0 ? 5 : weightSum;`” In this case, if all weights are zero, the algorithm defaults to:

$$U(X_j) = \frac{\sum_{i=1}^5 1 \cdot u_i(x_{ij})}{5} \times 100\% \quad (12)$$

which represents a standard, unweighted average utility, preserving the relative ranking of hotels even when all sliders are set to zero.

## 6.2 Relative Normalization Singularity (Zero-Range Denominator)

**6.2.1 The Problem.** Relative normalization maps raw values to a  $[0, 1]$  range based on the local minimum and maximum values:

$$u_i(x_{ij}) = \frac{x_{\max} - x_{ij}}{x_{\max} - x_{\min}} \quad (13)$$

This formula assumes that  $x_{\max} > x_{\min}$ . However, if a city dataset contains only hotels with the exact same value for an attribute, or if the filtered subset in a hybrid mode reduces the candidate set to hotels with identical prices or distances, the range becomes zero ( $x_{\max} - x_{\min} = 0$ ). In this scenario, evaluating the utility leads to division by zero:

$$u_i(x_{ij}) = \frac{x_{\max} - x_{ij}}{0} \quad (14)$$

which results in ‘Infinity’ or ‘NaN’ values, destabilizing the ranking.

**6.2.2 The Resolution.** The normalization engine was updated to include conditional checks. For price and location normalization, the code checks if the maximum and minimum values are equal: “`javascript const normPrice = maxPrice === minPrice ? 1.0 : (maxPrice - hotel.price) / (maxPrice - minPrice); const normLoc = maxLoc === minLoc ? 1.0 : (maxLoc - hotel.location) / (maxLoc - minLoc);`” If  $x_{\max} = x_{\min}$ , the utility is set to 1.0 for all hotels. This mathematically represents the logic that if all hotels are identical in a given attribute, they all share the maximum possible utility (1.0) for that attribute, neutralizing its impact on the final ranking.

## 6.3 Match Score Overshooting and Precision Errors

**6.3.1 The Problem.** In early stages, developers noted that match scores occasionally overshoot the 100% boundary (e.g., rendering scores of 101% or 105%). This occurred due to inconsistencies in the normalization boundaries of the benefit metrics. For instance, if absolute normalization was calculated as:

$$u_{\text{stars}}(s_j) = \frac{s_j - 1}{4} \quad (15)$$

but a hotel in the database was erroneously recorded with a star rating of 6 stars, the resulting utility would be 1.25, pushing the final match score over 100

**6.3.2 The Resolution.** The implementation was secured by applying strict range boundaries and precision clamping. All raw normalized scores are clamped to the interval  $[0.0, 1.0]$ :

$$u_i(x) = \max(0.0, \min(1.0, u_i(x))) \quad (16)$$

Additionally, the final match percentage is calculated, clamped to  $[0, 100]$ , and rounded to the nearest integer: “`javascript const percentageScore = Math.min(100, Math.max(0, Math.round((rawScore / totalWeight) * 100)));`” This ensures that the output is always an integer between 0% and 100%, preventing any visual anomalies in the user interface.

## 7 Future Work

While the current prototype demonstrates the feasibility of user-controlled, transparent tourism recommenders, several research and development paths remain open.

### 7.1 The Hybrid Algorithmic Approach

A natural evolution of the two modes is a hybrid algorithm that combines the strengths of both. Currently:

- Weighted MAUT is compensatory, which can lead to high cognitive load as users must constantly balance multiple sliders to filter out unacceptable options.



- Constraint-Based Filtering is non-compensatory and does not rank the matching items, which can leave the user with too many options or an empty set.

A hybrid recommender would operate as a two-stage pipeline:

- (1) **Stage 1: Constraint Filtering (Non-Compensatory screening):** The system applies user-defined constraints to filter out hotels that are completely unacceptable (e.g., price > \$250, or distance > 10 km). This reduces the candidate pool to a set of feasible options.
- (2) **Stage 2: Weighted MAUT (Compensatory ranking):** The system applies the user's preference weights to rank only the hotels that passed Stage 1.

This hybrid approach represents a realistic model of human decision-making, where decision-makers first eliminate unacceptable alternatives and then optimize among the survivors. It maintains full transparency and complies with the DSA, while reducing user cognitive load.

## 7.2 Proposed User Evaluation Protocol

To evaluate the user experience (UX) and the effectiveness of the explainability features, we propose a controlled user study protocol.

**7.2.1 Hypotheses.** We formulate three primary hypotheses:

- **H1 (Perceived Control):** Users interacting with the Weighted MAUT or Constraint Filtering interfaces report a higher sense of control over recommendations compared to a baseline "black-box" interface.
- **H2 (System Trust):** Exposing the collapsible mathematical breakdown panels increases user trust in the system's recommendations without causing information overload.
- **H3 (DSA Compliance Impact):** The presence of the global explanation banner and KaTeX formulas increases user comprehension of the algorithm's inner workings.

**7.2.2 Experimental Design.** We propose a within-subjects laboratory study with  $N = 30$  participants. Each participant will perform travel planning tasks under three experimental conditions:

- (1) **Condition 1 (Baseline - Black Box):** A traditional recommender interface displaying hotel cards ranked by a hidden algorithm, with no sliders and no explanation panels.
- (2) **Condition 2 (Weighted MAUT with On-Demand Explanations):** The interactive slider-based interface with collapsible math breakdown panels.
- (3) **Condition 3 (Constraint Filtering with Explanations):** The constraint-based interface showing absolute threshold sliders.

The ordering of conditions will be counterbalanced to prevent learning effects. Following each condition, participants will complete standardized questionnaires to measure:

- **System Usability:** Measured via the System Usability Scale (SUS).
- **User Experience & Trust:** Measured using the ResQue (Recommender Systems Quality of Experience) framework, focusing on trust, perceived accuracy, control, and transparency.
- **Cognitive Load:** Measured via the NASA-TLX (Task Load Index) to evaluate if exposing mathematical details increases mental demand.

The results of this study will guide the refinement of the UI, helping to strike a balance between complete mathematical transparency and user cognitive ease.

## 8 Conclusion

The era of black-box recommender systems is facing both user-experience and regulatory challenges. This paper presented an interactive, transparent tourism recommender system prototype designed to address the

compliance requirements of the EU Digital Services Act (DSA). By implementing both a compensatory Weighted Multi-Attribute Utility Theory (MAUT) model and a non-compensatory Constraint-Based Filtering model, the prototype provides users with direct, profiling-free control over recommendation logic. We integrated tiered explainability features, combining simple plain-language summaries with on-demand, KaTeX-rendered mathematical breakdowns. Finally, we analyzed and resolved mathematical edge cases, including division-by-zero singularities and score overshooting, ensuring a stable and robust system. This architecture serves as a blueprint for developers seeking to combine rich user control, modern web aesthetics, and strict regulatory compliance in recommender system design.

## Acknowledgments

The author would like to thank the teaching staff of the Lab Course: Projects in Recommender Systems (SS 2026) at the Chair of Connected Mobility, Technical University of Munich, for their guidance and feedback throughout the project.

## References

- [1] Gediminas Adomavicius and Alexander Tuzhilin. 2005. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering* 17, 6 (2005), 734–749. doi:10.1109/TKDE.2005.99
- [2] Joan Borras, Antonio Moreno, and Aida Valls. 2014. Intelligent tourism recommender systems: A survey. *Expert Systems with Applications* 41, 16 (2014), 7370–7389. doi:10.1016/j.eswa.2014.06.007
- [3] European Parliament and Council of the European Union. 2022. Regulation (EU) 2022/2065 of the European Parliament and of the Council of 19 October 2022 on a Single Market For Digital Services and amending Directive 2000/31/EC (Digital Services Act). <http://data.europa.eu/eli/reg/2022/2065/oj>.
- [4] Mohamed Elyes Ben Haj Kbaier, Hela Masri, and Saoussen Krichen. 2017. A Personalized Hybrid Tourism Recommender System. In *2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA)*. 244–250. doi:10.1109/AICCSA.2017.12
- [5] Ralph L. Keeney and Howard Raiffa. 1993. *Decisions with Multiple Objectives: Preferences and Value Trade-Offs*. Cambridge University Press, Cambridge, UK.
- [6] Bart P. Knijnenburg, M. C. Willemsen, Z. Gantner, H. Soncu, and C. Newell. 2012. Explaining the user experience of recommender systems. *User Modeling and User-Adapted Interaction* 22, 4-5 (2012), 441–504. doi:10.1007/s11257-011-9118-4
- [7] Sean M. McNee, John Riedl, and Joseph A. Konstan. 2006. Being accurate is not enough: How accuracy metrics have hurt recommender systems. (2006), 1097–1101. doi:10.1145/1125451.1125659
- [8] Francesco Ricci. 2011. Travel recommender systems. In *Recommender Systems Handbook*. Springer, 527–556. doi:10.1007/978-0-387-85820-3\_17
- [9] Joy Lal Sarkar, Abhishek Majumder, Chhabi Rani Panigrahi, Sudipta Roy, and Bibudhendu Pati. 2023. Tourism recommendation system: a survey and future research directions. *Multimedia Tools and Applications* 82, 6 (2023), 8983–9027. doi:10.1007/s11042-022-12167-w
- [10] Christian Schmitt, Dietmar Dengler, Michael Bauer, et al. 2002. The MAUT-Machine: An Adaptive Recommender System. In *Proceedings of the ABIS Workshop* (Hannover, Germany).
- [11] Kirsten Swearingen and Rashmi Sinha. 2001. Beyond algorithms: An HCI perspective on recommender systems. (2001).
- [12] Nava Tintarev and Judith Masthoff. 2012. Evaluating recommendation explanations: A survey. *User Modeling and User-Adapted Interaction* 22, 4-5 (2012), 399–439. doi:10.1007/s11257-011-9106-8

## A Complete Hotel Database

Table 2 presents the complete dataset of 20 hotels used in the recommender system prototype, split between Vienna (v1–v10) and Munich (m1–m10).

## B Author Information and Contribution Details

In accordance with course requirements, Table 3 documents the academic details, contact endpoints, and contributions of the group member.

Table 2. Hotel Database Attributes

| ID                   | Hotel Name                      | Price (\$/night) | Distance (km) | Stars | Rating (/10) | Transport (/10) |
|----------------------|---------------------------------|------------------|---------------|-------|--------------|-----------------|
| <b>Vienna Hotels</b> |                                 |                  |               |       |              |                 |
| v1                   | Grand Imperial Vienna           | 350              | 0.5           | 5     | 9.6          | 9.5             |
| v2                   | Boutique Hotel am Stephansplatz | 180              | 0.1           | 4     | 9.2          | 10.0            |
| v3                   | Danube Riverside Hotel          | 95               | 4.5           | 3     | 8.0          | 6.5             |
| v4                   | Schönbrunn Palace Suites        | 290              | 6.0           | 5     | 9.8          | 8.0             |
| v5                   | Hostel Vienna West              | 35               | 3.2           | 1     | 7.5          | 9.0             |
| v6                   | Art Deco Hotel Prater           | 140              | 2.8           | 4     | 8.7          | 8.5             |
| v7                   | Economy Inn Vienna              | 60               | 5.5           | 2     | 6.8          | 5.0             |
| v8                   | Museum Quarter Retreat          | 210              | 1.2           | 4     | 9.4          | 9.0             |
| v9                   | Green Oasis Hotel               | 110              | 7.5           | 3     | 8.9          | 4.0             |
| v10                  | The Ritz-Carlton Vienna         | 450              | 0.8           | 5     | 9.7          | 10.0            |
| <b>Munich Hotels</b> |                                 |                  |               |       |              |                 |
| m1                   | Bayerischer Hof                 | 420              | 0.4           | 5     | 9.5          | 9.5             |
| m2                   | Marienplatz Business Hotel      | 160              | 0.2           | 4     | 8.8          | 10.0            |
| m3                   | Oktoberfest Lodge               | 85               | 2.5           | 2     | 7.2          | 8.0             |
| m4                   | Englischer Garten Retreat       | 250              | 3.0           | 4     | 9.3          | 7.0             |
| m5                   | Schwabing Youth Hostel          | 40               | 4.0           | 1     | 8.1          | 8.5             |
| m6                   | Olympic Park Hotel              | 130              | 6.5           | 3     | 8.5          | 7.5             |
| m7                   | Airport Express Inn             | 90               | 28.0          | 3     | 7.8          | 9.0             |
| m8                   | Viktualienmarkt Boutique        | 190              | 0.6           | 4     | 9.1          | 9.5             |
| m9                   | Isar River View                 | 210              | 1.8           | 4     | 8.9          | 6.0             |
| m10                  | Mandarin Oriental Munich        | 550              | 0.5           | 5     | 9.8          | 9.5             |

Table 3. Author Identification and Contribution Table

| Full Name (as on transcript) | Email Address       | GitHub Handle | Key Contributions                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------|---------------------|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| David Gallob                 | david.gallob@tum.de | Daev          | Full design and implementation of the React application;<br>formulation and integration of the Weighted MAUT algorithm;<br>formulation and coding of the Constraint-Filtering logic;<br>implementation of KaTeX explanations and UI styling;<br>resolution of mathematical singularities and edge cases;<br>drafting of the final scientific paper and repository disclosure |

## C Generative AI Disclosure

This project utilized Generative AI tools (specifically Google DeepMind’s Gemini 3.5 Flash) during development. The AI was employed as a pair-programming assistant to:

- Draft and optimize the user interface layouts and CSS styling rules.
- Debug mathematical singularities (such as relative normalization division by zero) and recommend JavaScript state handling best practices.

- Assist in structuring and editing the LaTeX code of this scientific report.

All design decisions, algorithmic formulations, bug resolutions, and report text were audited, verified, and approved by the human author to ensure compliance with the scientific standards of the Technical University of Munich.